



BUPA Chile · QA Playbook

Marco de trabajo QA · Metodología SDD · Pipeline CI/CD · Automatización n8n

QA Lead: Jaime Quiñelen Villar

BUPA Chile · Portal Pacientes

v1.0 · Mayo 2026

MARCO DE TRABAJO

Proceso QA — 7 Fases

Flujo completo de aseguramiento de calidad desde la recepción del requerimiento hasta el cierre con métricas gerenciales. El QA Lead lidera cada fase, coordinando con el equipo de desarrollo y los stakeholders.

F1

Requerimiento

QA Lead · Jira + Metodología SDD

SDD

Jira

Entrada al proceso

ACTIVIDADES DEL QA LEAD

- Recibe requerimiento de stakeholders o PM
- Crea **Epic** y **Story** en Jira con descripción funcional completa
- Documenta **SDD completo** (9 secciones) en la descripción del ticket: objetivo, alcance, actores, flujos, criterios de aceptación, riesgos, datos de prueba, entornos y restricciones
- Activa el workflow de generación automática del PDF del SDD y notificación al equipo
- Avanza el ticket en el board de Jira



ARTEFACTOS PRODUCIDOS

ENTREGABLES

SDD.md (9 secciones)

Jira Epic

Jira Story

PDF del SDD

HERRAMIENTAS

Jira

n8n (WF generación SDD)

Gmail



CASOS DE PRUEBA

- Define casos de prueba con nomenclatura estándar:

TIPO	CÓDIGO	DESCRIPCIÓN
Happy Path	TC-XXX-FP	Flujo principal exitoso del requerimiento
Edge Case	TC-XXX-EC	Casos límite, errores y comportamientos alternativos

PLAN DE PRUEBAS MANUALES

- **Flujos principales** que requieren verificación humana (visual, experiencia de usuario)
- **Edge cases** que requieren configuración especial de ambiente (DevOps, throttling de red, estados simulados)

PLAN DE PRUEBAS AUTOMATIZADAS — CYPRESS

- Identifica los TCs automatizables (flujos deterministas y repetibles)
- Define **criterios PASS/FAIL explícitos** por TC antes de escribir el spec
- Carga **Xray** : TestPlan + TestExecution por requerimiento TCs en

ARTEFACTOS

- Test-Plan.md
- TCs en Xray
- Criterios PASS/FAIL
- Matriz REQ vs TC

SDD EN PROCESO

 →

READY TO DEV



F3

Desarrollo + Shift-Left

Dev Team implementa · QA Lead revisa
cobertura por categoría

Shift-Left

4 Categorías

Dev → QA aprueba → PR

ROL DEL QA LEAD — REVISIÓN SHIFT-LEFT

- El equipo de desarrollo implementa la funcionalidad y escribe pruebas unitarias
- El QA **revisa la cobertura de unit tests por categoría** antes de permitir el Code Review
- Si la cobertura es insuficiente, solicita más tests antes de avanzar
- QA aprueba → Code Review → PR abierto → pipeline se activa automáticamente

READY TO DEV



CODE REVIEW/PR

REVISIÓN POR CATEGORÍA**UX**

- ¿Tests cubren flujos de navegación del usuario?
- ¿Hay test de secuencias de interacción (Tab, focus)?

UI

- ¿Tests de render de componentes visuales?
- ¿Tests de atributos ARIA y contraste?

FRONTEND

- ¿Tests del bootstrap del framework (Angular)?
- ¿Tests de lógica de componentes y validaciones?

BACKEND

- ¿Tests de respuesta API y contratos?
- ¿Tests de seguridad (SSL, HTTPS, redirect)?



F4

Pipeline CI/CD

GitHub Actions · Cypress · Trigger: push/PR a ramas principales

7 Stages

GitHub Actions

Cypress

STAGES DEL PIPELINE

Stage 1 Instalación de dependencias del proyecto **npm ci**

Stage 2 Análisis estático de código (linting) **ESLint**

Stage 3 E2E Smoke Tests — flujo principal de cada REQ **Cypress**

Stage 4 E2E Flujos Críticos — casos de mayor riesgo por REQ **Cypress**

Stage 5 E2E Mobile — viewport reducido (solo rama develop) **Cypress**

Stage 6 Notificación del resultado al sistema de workflows **n8n webhook**

Stage 7 Reporte de resultados en GitHub Step Summary **GitHub Actions**

RESULTADO DEL PIPELINE**FALLO**

El workflow de alertas notifica automáticamente por email, mensajería y abre un defecto en Jira con prioridad máxima

ÉXITO

El pipeline notifica el resultado y el proceso continúa hacia la Fase 5 — Test UAT

ARTEFACTOS GENERADOS

Screenshots de fallos

Videos de ejecución

GitHub Step Summary

Notificación n8n



F5

Test UAT

QA Lead · Ambiente staging · Pruebas manuales y automatizadas

Staging

Cypress

Manual + Automatizado

PRUEBAS MANUALES POR CATEGORÍA**UX**

- Exploración de flujos, estados vacíos, usabilidad

UI

- Revisión visual vs diseño de referencia (Figma)

FRONTEND

- Validaciones en navegador, comportamiento del framework

BACKEND

- Validación de API, contratos y seguridad SSL

- Resultado manual → registra en Xray → workflow de estado diario notifica al equipo

PRUEBAS AUTOMATIZADAS — CYPRESS

- **Smoke Tests** — flujo principal por REQ → reporte HTML → email automático
- **Flujos Críticos** — casos de mayor riesgo → alerta inmediata si falla (Jira + email + mensajería)
- **Mobile** — viewport reducido → reporte HTML → email automático

BUG ENCONTRADO

Abre defecto en Jira · El proceso regresa a la Fase 3

APROBADO

Obtiene **VoB Usuario** (firma funcional) + **Ok Técnico** (firma TI) · Avanza a Fase 6

CODE REVIEW/PR →

TEST UAT →

VOB + OK TI

F6

Test Producción

QA Lead · Ambiente PROD · Solo flujo principal SDD + flujos críticos

Solo PROD

Flujo principal + Críticos

Monitoreo continuo

ALCANCE EN PRODUCCIÓN

- Se **únicamente** los casos E2E del flujo principal del SDD (Happy Path)
- Se **flujos** de mayor riesgo ejecutan **críticos** funcional definidos por los REQ
- **No se** que requieran configuración **ejecutan** especial de ambiente en **edge cases** PROD
- Workflow **health** cada hora verifica de **check** disponibilidad de **automático** ambientes y alerta si hay caída
- Workflow **resultado** notifica a QA Lead de **post-pipeline** por email y mensajería



RESULTADO EN PRODUCCIÓN

DEFECTO EN PROD — ESCAPED DEFECT
Se registra como defecto escapado con clasificación **found-in: PROD** en Jira · Se abre incidente urgente · Impacta la métrica de Escaped Defects

APROBADO
Ticket avanza a DONE · El requerimiento queda cerrado · Se alimentan las métricas semanales de la Fase 7



F7

Cierre y Métricas

QA Lead · Dashboard semanal · 5 KPIs gerenciales

5 KPIs

Dashboard semanal

Reporte a gerencia

ACTIVIDADES DEL QA LEAD

- El **dashboard** se ejecuta automáticamente al cierre del sprint
- Extrae datos de Jira (bugs, resolución) y GitHub (resultados del pipeline)
- Genera el **5 KPIs** y lo envía a los **gerenciales** gerencia por email
- El QA Lead revisa las métricas, identifica tendencias y define acciones para el siguiente sprint
- El **se** desde la Fase 1 para el ciclo **repite** siguiente requerimiento o sprint

5 KPIS GERENCIALES

KPI	QUÉ MIDE
Pass/Fail Rate	Porcentaje de TCs que pasan vs fallan en cada ejecución
Test Coverage	% de requerimientos cubiertos por al menos un caso de prueba
Automation Coverage	% de casos de prueba automatizados sobre el total definido
Escaped Defects	Defectos que llegaron a PROD sin ser detectados en UAT
Defect Resolution	Tiempo promedio de resolución de defectos vs SLA por severidad

MÉTRICAS GERENCIALES

5 KPIs de Calidad

Indicadores clave que el QA Lead reporta a gerencia en cada cierre de sprint. Los porcentajes se actualizan semanalmente vía dashboard automatizado (WF de reporte semanal).

PASS / FAIL RATE

0%

Porcentaje de casos de prueba que pasan sobre el total ejecutado. **Target: ≥ 95%** — Se calcula automáticamente al finalizar cada ejecución de Cypress.

TEST COVERAGE

0%

% de requerimientos del sprint cubiertos por al menos un caso de prueba. **Target: ≥ 90%** — Calculado desde la matriz REQ vs TC en Xray.

AUTOMATION COVERAGE

0%

% de TCs automatizados sobre el total definido. **Target: ≥ 70%** — Fórmula: (TCs automatizados / total TCs) x 100.

ESCAPED DEFECTS

0

Defectos que llegaron a PROD sin detectarse en UAT.

Target: 0 — Clasificados con found-in: PROD en Jira. Cada uno impacta el SLA.

DEFECT RESOLUTION

0%

% de defectos resueltos dentro del SLA. **Target: ≥ 95%**

— SLA: Critical 1d · High 2d · Medium 5d · Low 10d.

STACK TECNOLÓGICO

Herramientas del Proceso QA

Conjunto de herramientas que sostienen el proceso de calidad. Cada una cumple un rol específico en el ciclo QA.



Cypress

Framework de Automatización E2E

Ejecuta pruebas end-to-end en navegador real (Chrome, Firefox, Edge). Permite automatizar flujos completos de usuario, validar la UI, tomar screenshots y grabar videos de cada ejecución. Se integra nativamente con GitHub Actions en el pipeline.

Smoke Tests

Flujos Críticos

Mobile Testing

Screenshots / Videos



cypress-axe

Plugin de Accesibilidad WCAG

Integración de axe-core dentro de Cypress. Valida automáticamente el cumplimiento de estándares WCAG 2.1 niveles A y AA: contraste de colores, atributos ARIA, orden de foco, etiquetas accesibles y más.

WCAG 2.1 A/AA

Contraste

ARIA

Accesibilidad

**n8n**

Plataforma de Automatización de Workflows

Motor de automatización que conecta todas las herramientas del proceso QA. Ejecuta workflows al recibir eventos (webhooks), genera reportes HTML, envía notificaciones y consolida métricas de múltiples fuentes sin intervención manual.

Reportes automáticos

Notificaciones

Integración de APIs

Dashboards

**GitHub Actions**

Motor CI/CD

Ejecuta el pipeline QA automáticamente ante cada push o PR. Orquesta los stages de instalación, linting, pruebas E2E y notificaciones. Genera el Step Summary con el resultado de cada ejecución y almacena artefactos (screenshots, videos).

Pipeline CI/CD

Artefactos

Step Summary

Secrets

**Jira**

Gestión de Requerimientos y Defectos

Centraliza la gestión de requerimientos (Epics, Stories) y el tracking de defectos. El board QA implementa el flujo de estados completo desde BACKLOG hasta DONE. Los workflows de n8n interactúan con la API de Jira para abrir bugs, extraer métricas y enviar reportes.

Epics / Stories

Bug tracking

Board Kanban

Métricas

**Xray**

Gestión de Pruebas (plugin Jira)

Plugin de Jira que provee trazabilidad completa entre requerimientos y casos de prueba. Permite crear TestPlan y TestExecution, registrar resultados de ejecuciones manuales y calcular la cobertura de pruebas por requerimiento.

TestPlan

TestExecution

Trazabilidad REQ→TC

Coverage

AUTOMATIZACIÓN N8N**Workflows de Soporte QA**

Conjunto de workflows automatizados que operan en paralelo al proceso QA. Eliminan tareas manuales repetitivas, garantizan notificaciones en tiempo real y generan reportes sin intervención del QA Lead.

Reporte de Regresión Post-Ejecución

Fase 4 / 5

Se activa al finalizar cada ejecución de Cypress. Genera un reporte HTML con los resultados (PASS/FAIL por TC) y lo envía por email al QA Lead. Diferencia entre ejecuciones de smoke, flujos críticos y mobile.

Post-ejecución Cypress

Herramientas: Cypress · email-server · Gmail

Resumen Diario de Bugs Abiertos

Fase 5 / 6

Extrae todos los defectos abiertos de Jira cada mañana, los consolida en una planilla Excel y envía el resumen por email. Permite al QA Lead comenzar el día con visibilidad total del estado de defectos.

Diario · 9:00 AM

Herramientas: Jira API · Excel · Gmail

Monitoreo de Nuevos Tickets Jira

Fase 5

Detecta nuevos tickets creados en el board de Jira y notifica al equipo en tiempo real. Garantiza que ningún defecto abierto pase desapercibido durante el día.

Cada 2 minutos

Herramientas: Jira API · Excel · Gmail

Estado Diario QA

Fase 5 / 7

Genera y envía un resumen del estado del día al cierre de jornada: bugs abiertos, TCs ejecutados, incidencias y avance del sprint. Mantiene informado al equipo sin reuniones adicionales.

Diario · 6:00 PM

Herramientas: Jira API · Gmail

Health Check de Ambientes

Fase 5 / 6

Verifica la disponibilidad de los ambientes QA (staging y producción) de forma continua. Si detecta un ambiente caído, alerta al QA Lead y al equipo de DevOps de inmediato.

Cada hora

Herramientas: HTTP check · Gmail · Mensajería

Generación de SDD PDF

Fase 1

Al crear o actualizar un ticket de Jira con el SDD documentado, genera automáticamente el PDF del documento y lo envía al equipo por email y mensajería. Mantiene la documentación siempre actualizada y accesible.

Al crear/actualizar ticket Jira

Herramientas: Jira API · PDF generator · Gmail · WhatsApp

Alerta de Fallos Críticos

Fase 4 / 5

Detecta fallos en los flujos críticos de Cypress y actúa de inmediato: abre un defecto en Jira con prioridad máxima (Highest), notifica al QA Lead y al equipo de desarrollo por email y mensajería.

En tiempo real · Post-pipeline

Herramientas: Cypress · Jira API · Gmail · Mensajería

Resultado del Pipeline CI/CD

Fase 4

Recibe el resultado de cada ejecución del pipeline de GitHub Actions vía webhook. Notifica al QA Lead con el estado (success/failure), rama, commit y enlace al run para trazabilidad completa.

Post-pipeline · Webhook

Herramientas: GitHub Actions · n8n webhook · Gmail · Mensajería · Jira

Dashboard Semanal QA

Fase 7

Al cierre del sprint genera automáticamente el reporte semanal con los 5 KPIs gerenciales: extrae datos de Jira y GitHub, calcula los indicadores y envía el informe ejecutivo a gerencia por email.

Semanal · Cierre de sprint

Herramientas: [Jira API](#) · [GitHub API](#) · [Gmail](#)

DECISION DE RELEASE

Quality Gates — Go / No-Go

Criterios objetivos que el QA Lead evalúa antes de autorizar cada release. Si todos los gates están en verde, se emite el GO con evidencia documentada. Un solo gate en rojo detiene el release.

CRITERIOS DE RELEASE

CRITERIO	UMBRAL MÍNIMO	ESTADO REQUERIDO
Casos ejecutados	≥ 90%	Aprobado en Xray
Bugs críticos abiertos	0	Cerrado o riesgo aceptado
Smoke Tests UAT	100% pass	Cypress verde
Pipeline CI/CD	Sin errores	GitHub Actions verde
VoB Usuario	Requerido	Firmado por responsable funcional
Ok Técnico	Requerido	Firmado por responsable TI

DECISION FINAL

GO — RELEASE AUTORIZADO

Todos los gates en verde. El QA Lead emite el GO por escrito con evidencia adjunta (screenshots Cypress, resumen Xray, firmas VoB y Ok TI). Se documenta en Jira y se avanza a DONE.

NO-GO — RELEASE BLOQUEADO

Al menos un gate en rojo. El QA Lead documenta el bloqueo específico, el responsable de resolverlo y la fecha estimada de resolución. El release queda suspendido hasta nuevo GO.

CRITERIOS DE CIERRE

Definition of Done — QA

Un requerimiento solo se marca como DONE cuando cumple todos estos criterios. No hay excepciones sin aprobación formal del QA Lead.



Documentación

SDD + Jira

- ✓ SDD completo (9 secciones) en Jira
- ✓ Casos de prueba documentados con criterios PASS/FAIL
- ✓ Matriz REQ vs TC actualizada en Xray
- ✓ PDF del SDD generado y enviado al equipo



Ejecución

UAT + Cypress

- ✓ Pruebas manuales ejecutadas en UAT con evidencia
- ✓ Smoke tests Cypress pasando en UAT
- ✓ Flujos críticos automatizados y en verde
- ✓ Bugs críticos y altos cerrados o riesgo aceptado



Aprobaciones

VoB + Ok TI

- ✓ VoB Usuario firmado por responsable funcional
- ✓ Ok Técnico firmado por responsable TI
- ✓ Ticket avanzado a DONE en Jira
- ✓ Métricas actualizadas en dashboard semanal

TIEMPOS DE RESPUESTA

SLA por Severidad de Defecto

Tiempo máximo de resolución desde la apertura del defecto en Jira. El incumplimiento del SLA impacta directamente el KPI de Defect Resolution.

SEVERIDAD	DEFINICIÓN	SLA RESOLUCIÓN	RESPONSABLE
Critical	Bloquea el flujo principal — el portal no carga o login falla	1 día hábil	Dev + DevOps
High	Funcionalidad crítica degradada — comportamiento incorrecto severo	2 días hábiles	Dev
Medium	Funcionalidad afectada pero con workaround disponible	5 días hábiles	Dev
Low	Problema cosmético o de bajo impacto para el usuario	10 días hábiles	Dev